Eleanor Schema Documentation

Table of contents

Overview

The primary function of **Eleanor** is to accept a problem detailing how to select fully-specified aqueous speciation problems, referred to as an "order", equilibrate them, and curate the resulting data.

Each such selected problem corresponds to a point (row) in the "variable space" (`variable_space` table). Each order may then have any number of associated variable space rows. The properties of the variable space points are broken out over a number of tables, e.g. the `elements` table contains the elemental composition of the fluid before speciation, the `mineral_reactants` table contains the amount and titration rate of each mineral to be reacted with the system, etc…

Speciation is then handled by the kernel specified in the order, which may produce zero or more points (rows) in the "equilibrium space" (`equilibrium_space` table). Each row for a given variable space point represents some form or degree of equilibration. For example, the `eq36` kernel performs the equilibration in two stages, first speciating the fluid alone (the `"eq3"` stage) and then equilibrating the fluid with the reactants (the `"eq6"` stage). As such, any variable space point equilibrated with the `eq36` kernel will have at least two equilibrium points associated with it (more on this below). As with variable space points, several tables contain detailed information about the equilibrium space points, e.g. the `equilibrium_elements` table contains the elemental composition of the fluid after the given speciation, the `equilibrium_reactants` table contains information about how much of each reactant has been, and how much remains to be reacted, etc…

Tables

This document describes each table in the schema and its relationship to the other tables. For brevity we use a few abbreviations as follows:

Key	
PK	Primary Key
FK	Foreign Key
N	Nullable

Every table a has single primary key (**PK**)—`id`—without exception, though that my change in subsequent versions of **Eleanor** (but probably not).

As described in Overview, the **Eleanor** database is somewhat hierarchical with the `orders` table as the root, zero or more entries in the `variable_space` table referring to an entry in `orders`, and with zero or more entries in the `equilibrium_space` table referring to an entry in the `variable_space` table. These relationships are reflected in two ways in this document:

1. The document headings are structured such that each table refers to the table one-level above it.
2. The foreign keys (**FK**) are noted in the tables, and the associated column name links to the referenced table. These column names are generally of the form `*_id`. The main exceptions are one-to-one tables where `id` is both the primary key and the foreign key, e.g. `kernel` and `scratch`, which each reuse `variable_space` ids.

Most columns in the **Eleanor** database are `NOT NULL`, meaning they are guaranteed to have a value, though that value may be some standard value representing "absent" or "missing", e.g. `-inf`. Columns that are nullable (**N**) are denoted as such.

orders

PK	FK	N	Column Name	Type	Description	Unit
✓			`id`	`INTEGER`	The order identifier	
			`name`	`VARCHAR`	The name of the order	
			`tags`	`TEXT[]`	Tags for organizing orders (default: `'{}'`)	
			`eleanor_version`	`VARCHAR`	The version of eleanor used to run the order	
			`raw`	`JSONB`	The raw, JSON-encoded contents of the order	
			`create_date`	`DATETIME`	The datetime at which the order was created	

When the user runs an order, **Eleanor** stores an entry in the `orders` table which includes the autoincremented `id`, the name of the order (extracted from the YAML/TOML/JSON file), optional `tags` for organization, and the version of **Eleanor** used to execute the order.

Indexes:

- INDEX `orders_name_idx` on (`name`)
- INDEX `orders_tags_idx` on (`tags`) USING GIN

The `raw` column stores the original content of the order file in JSON format. It does not carry an `id`: the identifier belongs to this table's `id` column, assigned by the sequence when the row is inserted, and an order file may not declare one.

The `create_date` value is the datetime at which the order was initially created.

Example:

```
SELECT id, name, tags, eleanor_version, create_date FROM orders;

+----+----------------------+--------+-----------------+------------------------+
| id | name                 | tags   | eleanor_version | create_date            |
|----+----------------------+--------+-----------------+------------------------|
| 1  | A Wild Example Order | {test} | 0.19.0          | 2026-01-09 10:59:01.98 |
+----+----------------------+--------+-----------------+------------------------+
```

variable_space

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The variable space point identifier	
	✓		order_id	INTEGER	The order id for which the point was generated	
			water_mass	DOUBLE	The mass of water in the system	kg
			temperature	DOUBLE	The temperature	$^{\circ}C$
			pressure	DOUBLE	The pressure	bar
			exit_code	INTEGER	The exit code generated by the kernel (non-zero for failure)	
		✓	error	TEXT	The error message generated by the kernel on failure, if any	
			create_date	DATETIME	The datetime when the variable space point was created	
			start_date	DATETIME	The datetime when the equilibration started	
			complete_date	DATETIME	The datetime when the equilibration completed	

Indexes:

- INDEX `variable_space_order_id_exit_code_idx` on (`order_id`, `exit_code`)
- INDEX `variable_space_exit_code_failed_idx` on (`exit_code`) WHERE `exit_code <> 0`
- INDEX `variable_space_temperature_idx` on (`temperature`)
- INDEX `variable_space_pressure_idx` on (`pressure`)

When the user runs **Eleanor**, a specified number of variable space points are selected based on the constraints provided by that order. Each point represents the unspeciated initial state of a system and includes such information as:

- Water Mass
- Temperature
- Pressure
- Kernel Settings (`kernel`)
- Elemental Composition (`elements`)
- Species Constraints (`species`)
- Reactant Specifications, including:
  - Aqueous Reactants (`aqueous_reactants`)
  - Element Reactants (`element_reactants`)
  - Fixed Gas Reactants (`fixed_gas_reactants`)
  - Gas Reactants (`gas_reactants`)
  - Mineral Reactants (`mineral_reactants`)
  - Solid Solution Reactants (`solid_solution_reactants`)
  - Special Reactants (`special_reactants`)
- Suppressions (`suppressions`)

The `variable_space` table stores the core information about the variable space point per row. This includes an autoincrementing `id`, the `water_mass` (mass of water in the system in kilograms; defaults to 1.0 if unspecified in the order), the `temperature` and `pressure` of the system, and the datetime at which it was generated. Each point refers back to the order to which it belongs via the `order_id`.

After generating the variable space point, **Eleanor** proceeds to speciate the system using the user-specified kernel. The `exit_code` generated by the kernel as well as the datetimes of when the speciation started and completed are stored in the `variable_space` table as well. If the kernel fails (non-zero `exit_code`), the `error` column stores the associated error message.

Example:

```
SELECT * FROM variable_space ORDER BY id LIMIT 1;

+----+----------+------------+-------------+----------+-----------+-------+----------------------------+----------------------------
+----------------------------+
| id | order_id | water_mass | temperature | pressure | exit_code | error | create_date                    | start_date                     |
complete_date              |
|----+----------+------------+-------------+----------+-----------+-------+----------------------------
+----------------------------+----------------------------|
| 1  | 1        | 1.0        | 300.0       | 500.0    | 0         | <null>| 2026-01-09 10:59:02.541519 | 2026-01-09 10:59:02.546294 | 2026-01-09
10:59:02.843071 |
+----+----------+------------+-------------+----------+-----------+-------+----------------------------+----------------------------
+----------------------------+
```

scratch

PK	FK	N	Column Name	Type	Description	Unit
✓	✓		`id`	`INTEGER`	The scratch entry identifier corresponds to the variable space id	
			`zip`	`BYTEA`	A zip archive of the input files used and output files generated when equilibrating the variable space point. This includes a traceback when an error occurs	

The input files used and output files generated by the kernel when speciating a "variable space" point are stored in a `zip` archive in the `scratch` table. The `scratch` row will have the same `id` as the associated variable space point.

By default, **Eleanor** only stores scratch entries if an error occurs while speciating the variable space point, as signified by a non-zero exit code from the kernel. In such a case a `traceback.txt` file is included in the `zip` archive detailing the error. The user may specify, at runtime, that scratch be collected for all variable space speciations. In that case, no `traceback.txt` file is included in the `zip` archive.

Example:

```
SELECT * FROM scratch WHERE id = 1;

+----+----------+
```

```
| id | zip      |
|----+----------|
| 1  | <binary> |
+----+----------+
```

kernel

PK	FK	N	Column Name	Type	Description	Unit
✓	✓		id	INTEGER	The kernel entry identifier corresponds to the variable space point	
			type	VARCHAR	The type of the kernel used, e.g. eq36	
			settings	JSONB	A JSON-encoded object of the settings passed to the kernel	

Eleanor is designed to support user-provided kernels which may require specific configuration or settings. Those settings are stored in the `kernel` table, one row for each variable space point. The kernel settings row will have the same `id` as the variable space point to which it belongs. The `type` of the kernel is stored alongside a JSON encodedded `settings` object, the form of which is kernel-specific.

While, at the moment, only the `eleanor.kernel.eq36` kernel is provided out-of-the-box, future versions may include other kernel implementations or the user may design and implement their own. It is then difficult to design a schema for storing the settings. Using a JSON object makes things simpler given PostgreSQL's native JSON support.

Example:

```
SELECT * FROM kernel WHERE id = 1;

+----+------+-----------------------------------------------------+
| id | type | settings                                            |
|----+------+-----------------------------------------------------|
| 1  | eq36 | {"model": 0, "timeout": null, "basis_map": {}, ... } |
+----+------+-----------------------------------------------------+
```

elements

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The element identifier, specific to the variable space point	
	✓		variable_space_id	INTEGER	The id of the variable space point for which elemental molality was generated	
			name	VARCHAR	The name of the element (atomic symbol)	
			log_molality	DOUBLE	The log molality of the element in the variable space solution	$\log molal$

- INDEX elements_variable_space_id_name_idx on (variable_space_id, name)

The elemental composition of the system's fluid must be provided as part of the specification of a variable space point. Each element in the fluid is stored in the elements table with a unique, autoincremented id, the name of the element, and the log_molality that was chosen when the variable space was generated. Each element references the variable space point it is associated with by the variable_space_id.

Example:

```
SELECT *
 FROM elements
 WHERE variable_space_id = 1
 ORDER BY name;
```

```
+----+-------------------+------+--------------+
| id | variable_space_id | name | log_molality |
|----+-------------------+------+--------------|
| 1  | 1                 | C    | -9.0         |
| 4  | 1                 | Ca   | -9.0         |
| 5  | 1                 | Cl   | -9.0         |
| 7  | 1                 | Fe   | -9.0         |
| 6  | 1                 | Mg   | -9.0         |
| 8  | 1                 | N    | -9.0         |
| 2  | 1                 | Na   | -9.0         |
| 3  | 1                 | Si   | -9.0         |
+----+-------------------+------+--------------+
```

species

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The species identifier, specific to the variable space point	
	✓		variable_space_id	INTEGER	The id of the variable space point for which the species molality/activity/fugacity was generated	
			name	VARCHAR	The name of the species	
			value	DOUBLE	The log molality/activity/fugacity of the species in the variable space solution	variable

- INDEX species_variable_space_id_name_idx on (variable_space_id, name)

Constraints on the aqueous and gaceous species in the system may, though in some cases must, be provided as part of the specification of a variable space point, such as the pH and fO_2 of the system. Each species constraint is stored in the species table with a unique, autoincremented id, the name of the species, and the value of the constraint (log molality, log activity or log fugacity as appropriate) that was chosen when the variable space was generated. Each element references the variable space point it is associated with by the variable_space_id.

Example:

```
SELECT *
 FROM species
 WHERE variable_space_id = 1
 ORDER BY name;

+----+-------------------+-------+-------+
| id | variable_space_id | name  | value |
|----+-------------------+-------+-------|
| 1  | 1                 | H+    | -7.0  |
| 2  | 1                 | O2(g) | -1.0  |
+----+-------------------+-------+-------+
```

Reactants

Several types of reactant can be specified in an order, facilitating the simulation of a wide variety of systems. These types are:

- Aqueous Reactants (aqueous_reactants)
- Element Reactants (element_reactants)
- Fixed Gas Reactants (fixed_gas_reactants)
- Gas Reactants (gas_reactants)
- Mineral Reactants (mineral_reactants)
- Solid Solution Reactants (solid_solution_reactants)
- Special Reactants (special_reactants)

and each the details of reactants are stored in tables by type. However, they mostly follow the same basic form:

1. Each table has an autoincrementing `id`,
2. Each table has a `variable_space_id` that associates the entry with the variable space point,
3. The `name` of the reactant,
4. The molar amount of the reactant, `log_moles`, and
5. Either a `titration_rate` or `log_fugacity` as appropriate for the reactant type.

aqueous_reactants

PK	FK	N	Column Name	Type	Description	Unit
✓			`id`	`INTEGER`	The aqueous reactant identifier, specific to the variable space point	
	✓		`variable_space_id`	`INTEGER`	The id of the variable space point for which the aqueous reactant amount and titration rate were generated	
			`name`	`VARCHAR`	The name of the aqueous reactant	
			`log_moles`	`DOUBLE`	The log amount of the reactant to be reacted with the fluid	$\log mol$
			`titration_rate`	`DOUBLE`	The rate (in ξ) at which the reactant will be titrated into the fluid	mol/ξ

- INDEX `aqueous_reactants_variable_space_id_name_idx` on (`variable_space_id`, `name`)

An aqueous reactant models the addition (or removal) of a fixed molar quantity (`amount`) of an aqueous species into the system at a ξ-rate (`titration_rate`).

Example:

```
SELECT *
 FROM aqueous_reactants
 WHERE variable_space_id = 1
 ORDER BY name;

+----+-------------------+-------+-----------+----------------+
| id | variable_space_id | name  | log_moles | titration_rate |
|----+-------------------+-------+-----------+----------------|
| 1  | 1                 | CaCl2 | -2.0      | 1.0            |
+----+-------------------+-------+-----------+----------------+
```

element_reactants

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The element reactant identifier, specific to the variable space point	
	✓		variable_space_id	INTEGER	The id of the variable space point for which the element reactant amount and titration rate were generated	
			name	VARCHAR	The name of the element reactant	
			log_moles	DOUBLE	The log amount of the reactant to be reacted with the fluid	$\log mol$
			titration_rate	DOUBLE	The rate (in ξ) at which the reactant will be titrated into the fluid	mol/ξ

- INDEX element_reactants_variable_space_id_name_idx on (variable_space_id, name)

An element reactant is just a special case of special_reactant in which a fixed molar quantity (amount) of a single (unionized) element is added to the bulk composition of the fluid at some ξ-rate (titration_rate).

Example:

```
SELECT *
 FROM element_reactants
 WHERE variable_space_id = 1
 ORDER BY name;

+----+-------------------+------+-----------+----------------+
| id | variable_space_id | name | log_moles | titration_rate |
|----+-------------------+------+-----------+----------------|
| 1  | 1                 | Si   | -2.0      | 1.0            |
+----+-------------------+------+-----------+----------------+
```

fixed_gas_reactants

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The fixed gas reactant identifier, specific to the variable space point	
	✓		variable_space_id	INTEGER	The id of the variable space point for which the fixed gas reactant amount and fugacity were generated	
			name	VARCHAR	The name of the fixed gas reactant	
			log_moles	DOUBLE	The log amount of the fixed gas buffer	$\log mol$
			log_fugacity	DOUBLE	The log fugacity of will be titrated into the fluid	mol/ξ

- INDEX fixed_gas_reactants_variable_space_id_name_idx on (variable_space_id, name)

A fixed gas reactant models gas exchange with a large, external gas reservoir at a fixed fugacity. Rather than the gas being "titrated" in, the `log_fugacity` is enforced through mass exchange with a buffer with initial size `amount`.

Example:

```
SELECT *
 FROM fixed_gas_reactants
 WHERE variable_space_id = 1
 ORDER BY name;

+----+-------------------+--------+-----------+--------------+
| id | variable_space_id | name   | log_moles | log_fugacity |
|----+-------------------+--------+-----------+--------------|
| 1  | 1                 | CO2(g) | -0.3      | -3.5         |
+----+-------------------+--------+-----------+--------------+
```

gas_reactants

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The gas reactant identifier, specific to the variable space point	
	✓		variable_space_id	INTEGER	The id of the variable space point for which the gas reactant amount and titration rate were generated	
			name	VARCHAR	The name of the gas reactant	
			log_moles	DOUBLE	The log amount of the reactant to be reacted with the fluid	$\log mol$
			titration_rate	DOUBLE	The rate (in ξ) at which the reactant will be titrated into the fluid	mol/ξ

- INDEX gas_reactants_variable_space_id_name_idx on (variable_space_id, name)

A gas reactant models reacting a fixed molar quantity (amount) of a gaseous species with the fluid at some ξ-rate (titration_rate).

Example:

```
SELECT *
 FROM gas_reactants
 WHERE variable_space_id = 1
 ORDER BY name;

+----+-------------------+-------+-----------+----------------+
| id | variable_space_id | name  | log_moles | titration_rate |
|----+-------------------+-------+-----------+----------------|
| 1  | 1                 | N2(g) | -0.3      | 1.0            |
+----+-------------------+-------+-----------+----------------+
```

mineral_reactants

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The mineral reactant identifier, specific to the variable space point	
	✓		variable_space_id	INTEGER	The id of the variable space point for which the mineral reactant amount and titration rate were generated	
			name	VARCHAR	The name of the mineral reactant	
			log_moles	DOUBLE	The log amount of the reactant to be reacted with the fluid	$\log mol$
			titration_rate	DOUBLE	The rate (in ξ) at which the reactant will be titrated into the fluid	mol/ξ

- INDEX mineral_reactants_variable_space_id_name_idx on (variable_space_id, name)

A mineral reactant models reacting a fixed molar quantity (amount) of a pure mineral with the fluid at some ξ-rate (titration_rate).

Example:

```
SELECT *
 FROM mineral_reactants
 WHERE variable_space_id = 1
 ORDER BY name;

+----+-------------------+----------+-----------+----------------+
| id | variable_space_id | name     | log_moles | titration_rate |
|----+-------------------+----------+-----------+----------------|
| 1  | 1                 | hematite | -0.3      | 1.0            |
+----+-------------------+----------+-----------+----------------+
```

solid_solution_reactants

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The solid solution identifier, specific to the variable space point	
	✓		variable_space_id	INTEGER	The id of the variable space point for which the solid solution amount and titration rate were generated	
			name	VARCHAR	The name of the solid solution	
			log_moles	DOUBLE	The log amount of the reactant to be reacted with the fluid	$\log mol$
			titration_rate	DOUBLE	The rate (in ξ) at which the reactant will be titrated into the fluid	mol/ξ

- INDEX solid_solution_reactants_variable_space_id_name_idx on (variable_space_id, name)

A solid solution reactant models reacting a fixed molar quantity (`amount`) of a solid solution with the fluid at some ξ-rate (`titration_rate`). Each such solid solution must include the mole-fraction of each of its end members. The end member data is stored separately in the `solid_solution_reactant_end_members` table.

Example:

```
SELECT *
 FROM solid_solution_reactants
 WHERE variable_space_id = 1
 ORDER BY name;

+----+-------------------+------------+-----------+----------------+
| id | variable_space_id | name       | log_moles | titration_rate |
|----+-------------------+------------+-----------+----------------|
| 1  | 1                 | olivine-ss | -0.3      | 1.0            |
+----+-------------------+------------+-----------+----------------+
```

solid_solution_reactant_end_members

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The solid solution end member id, specific to the solid solution	
	✓		solid_solution_reactant_id	INTEGER	The id of the solid solution for which this is an end member	
			name	VARCHAR	The name of the end member	
			fraction	DOUBLE	The mole-fraction of the end member in the solid solution	unitless

- CONSTRAINT fraction_in_range CHECK (0.0 <= fraction AND fraction <= 1.0)
- INDEX solid_solution_reactant_end_members_ssr_id_name_idx on (solid_solution_reactant_id, name)

Each solid solution reactant has a fixed distribution of end members, represented by the mole-fraction of the total solid solution.

Example:

```
SELECT em.*
 FROM solid_solution_reactant_end_members AS em
 JOIN solid_solution_reactants AS ssr ON ssr.id = em.solid_solution_reactant_id AND variable_space_id = 1
 ORDER BY em.name;

+----+----------------------------+------------+----------+
| id | solid_solution_reactant_id | name       | fraction |
|----+----------------------------+------------+----------|
| 1  | 1                          | fayalite   | 0.6      |
| 2  | 1                          | forsterite | 0.4      |
+----+----------------------------+------------+----------+
```

special_reactants

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The special reactant identifier, specific to the variable space point	
	✓		variable_space_id	INTEGER	The id of the variable space point for which the special reactant amount and titration rate were generated	
			name	VARCHAR	The name of the special reactant	
			log_moles	DOUBLE	The log amount of the reactant to be reacted with the fluid	$\log mol$
			titration_rate	DOUBLE	The rate (in ξ) at which the reactant will be titrated into the fluid	mol/ξ

- INDEX special_reactants_variable_space_id_name_idx on (variable_space_id, name)

A special reactant models reacting a fixed molar quantity (amount) of a user-specified molecular composition with the fluid at some ξ-rate (titration_rate). Each special reactant must have include its stoichiometric composition, which is stored separately in the special_reactant_compositions table.

Example:

```
SELECT *
 FROM special_reactants
 WHERE variable_space_id = 1
 ORDER BY name;

+----+------------------+-----------------+-----------+----------------+
| id | variable_space_id | name            | log_moles | titration_rate |
|----+------------------+-----------------+-----------+----------------|
| 1  | 1                | CalciumHydroxide | -2.0      | 1.0            |
+----+------------------+-----------------+-----------+----------------+
```

special_reactant_compositions

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The special reactant composition identifier	
	✓		special_reactant_id	INTEGER	The id of the special reactant	
			element	VARCHAR	The name of the element in the special reactant	
			count	INTEGER	The stoichiometric count of the element in the special reactant	

- INDEX special_reactant_compositions_special_reactant_id_element_idx on (special_reactant_id, element)

Each special reactant has a fixed stoichiometric composition. Each element and its occurrence (count) in the reactant is stored in a row of the special_reactant_composition table.

Example:

```
SELECT c.*
 FROM special_reactant_compositions AS c
 JOIN special_reactants AS sr ON sr.id = c.special_reactant_id AND sr.variable_space_id = 1
 ORDER BY c.element;

+----+---------------------+---------+-------+
| id | special_reactant_id | element | count |
|----+---------------------+---------+-------|
| 1  | 1                   | Ca      | 1     |
| 3  | 1                   | H       | 2     |
| 2  | 1                   | O       | 2     |
+----+---------------------+---------+-------+
```

suppressions

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The suprression identifier	
	✓		variable_space_id	INTEGER	The id of the variable space point	
		✓	name	VARCHAR	The name of the species/phase to suppress, e.g. H_2O_2, methane, olivine	
		✓	type	VARCHAR	The type of species/phase to suppress, e.g. “solid solutions”	

- `CONSTRAINT suppressions_well_defined CHECK (name is not null or type is not null)`
- INDEX `suppressions_variable_space_id_idx` on (`variable_space_id`)

Users can specify, within an order, that some species or entire phase type should be suppressed from forming. When a phase type is specified, exceptions can be provided. Those exceptions are stored separately in the `suppression_exceptions` table.

Example:

```
SELECT * FROM suppressions WHERE variable_space_id = 1;

+----+-------------------+------------+---------+
| id | variable_space_id | name       | type    |
|----+-------------------+------------+---------|
| 1  | 1                 | METHANE    | <null>  |
| 2  | 1                 | antigorite | <null>  |
| 3  | 1                 | <null>     | mineral |
+----+-------------------+------------+---------+
```

suppression_exceptions

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The suppression exception identifier	
			name	VARCHAR	The name the species/phase to except, e.g. H2O2, methane, olivine	
	✓		suppression_id	INTEGER	The id of the suppression for which to except	

- INDEX suppression_exceptions_suppression_id_idx on (suppression_id)

Each exception is stored as a row in the suppression_exceptions table, referring back to the suppression by the suppression_id value.

Example:

```
SELECT se.*
 FROM suppression_exceptions AS se
 JOIN suppressions AS s ON s.id = se.suppression_id AND s.variable_space_id = 1
 ORDER BY se.name;

+----+-----------+----------------+
| id | name      | suppression_id |
|----+-----------+----------------|
| 1  | magnetite | 3              |
+----+-----------+----------------+
```

equilibrium_space

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The equilibrium space point identifier	
	✓	✓	variable_space_id	INTEGER	The id of the variable space point for which the equilibrium was found	
			stage	VARCHAR	The stage of the equilibration, e.g. "eq3", "eq6"	
		✓	log_xi	DOUBLE	The logarithm of the ξ-step at which this equilibrium was found	unitless
			temperature	DOUBLE	The temperature	$^{\circ}C$
			pressure	DOUBLE	The pressure	bar
			"pH"	DOUBLE	The pH on the NBS scale	unitless
			"log_fO2"	DOUBLE	The logarithm of the oxygen (O_2) fugacity	$\log bar$
			"Eh"	DOUBLE	The redox potential	V
			log_activity_water	DOUBLE	The logarithm of the activity of water	unitless
			log_ionic_strength	DOUBLE	The logarithm of the ionic strength of the solution	$\log molal$
			solute_mass	DOUBLE	The total mass of all dissolved species	g
			solvent_mass	DOUBLE	The mass of the solvent (water)	g
			solution_mass	DOUBLE	The mass of the solution	g
			tds	DOUBLE	The total dissolved solute (TDS); dissolved solute mass per kilogram of solution	mg/kg
		✓	reactant_mass_reacted	DOUBLE	The total mass reacted into the system up to this ξ step	g
		✓	reactant_mass_remaining	DOUBLE	The total mass remaining to be reacted	g
			start_date	DATETIME	The datetime when the equilibration started	
			complete_date	DATETIME	The datetime when the equilibration completed	
			custom_properties	JSONB	Less common solution properties and any non-standard equilibrium properties computed by the kernel, stored as a JSON object	

Indexes:

- INDEX equilibrium_space_variable_space_id_stage_idx on (variable_space_id, stage)
- INDEX equilibrium_space_temperature_idx on (temperature)
- INDEX equilibrium_space_pressure_idx on (pressure)
- INDEX equilibrium_space_ph_idx on (pH)
- INDEX equilibrium_space_log_fo2_idx on (log_fO2)
- INDEX equilibrium_space_eh_idx on (Eh)
- INDEX equilibrium_space_log_ionic_strength_idx on (log_ionic_strength)

- INDEX `equilibrium_space_solute_mass_idx` on (`solute_mass`)
- INDEX `equilibrium_space_solvent_mass_idx` on (`solvent_mass`)
- INDEX `equilibrium_space_tds_idx` on (`tds`)
- INDEX `equilibrium_space_reactant_mass_reacted_idx` on (`reactant_mass_reacted`)
- INDEX `equilibrium_space_reactant_mass_remaining_idx` on (`reactant_mass_remaining`)

When **Eleanor** speciates a fluid, it is the kernel's responsibility to perform the calculations and provide one or more equilibrium space points. Each of these points, consisting of identifying information (`id`, `variable_space_id`, `stage` and `log_xi`) and a wide range of system properties (e.g. `temperature`, `pressure`, `"pH"`, etc…), is stored as a row in the `equilibrium_space` table. Additional, information about the system, such as the quantity of reactants reacted and remaining (`equilibrium_reactants`), the elemental composition of the fluid (`equilibrium_elements`), the molecular composition of the fluid (`equilibrium_aqueous_species`), etc…, are stored in auxiliary tables that link back to the equilibrium space point via a foreign key reference on their `equilibrium_space_id` column.

For example, the builtin `eleanor.kernel.eq36.Kernel` will produce at least 2 equilibrium space points: one representing the speciation of the fluid alone (the `"eq3"` stage), and the final equilibrium after reacting the fluid with any reactants (the `"eq6"` stage). In the case that the user configured the kernel with the `track_paths: true` option, (many) more than 2 equilibrium space points will be produced: the `"eq3"` point, and some number of intermediate points between the `"eq3"` system and the final `"eq6"` representing a titration path. That is, each point along the titration path represents the equilibration of the fluid with an increasing quantity of reactant. The progress along the path is tracked by ξ (stored as `log_xi`). The `log_xi` value will be `NULL` for `"eq3"` stage points as ξ is undefined.

Different kernels, and different stages processed by a given kernel, may result in some columns being either `NULL` or `-inf` for one reason or another. This can cause confusion. Take for example the case of `log_xi` before. The `"eq3"` stage points will have `log_xi = NULL`, and the first point of a titration path produced in the `"eq6"` stage will have `log_xi = -inf` ($\xi = 0$ at the start of the path). However, if the user did not enable path tracking or set the kernel's print settings judiciously, then the first `"eq6"` stage point may have a finite `log_xi`. Other examples include:

1. Since mass is not transferred in the `"eq3"` stage, columns such as `reactant_mass_reacted` and `reactant_mass_remaining` will be `NULL`.

Less commonly queried solution properties (such as `mole_fraction_water`, `log_gamma_water`, `pe`, `Ah`, charge-balance quantities, solid mass and volume changes, etc.) are stored in the `custom_properties` column. Each kernel may also compute and store additional non-standard properties there.

Example (Without Paths):

```
SELECT
   id,
   variable_space_id,
   stage,
   log_xi,
   temperature,
   pressure,
   "pH",
   start_date,
   complete_date
 FROM equilibrium_space
 WHERE variable_space_id = 1
 ORDER BY stage, log_xi;

+----+------------------+-------+--------------------+-------------+----------+--------+-----------------------+------------------------+
```

id	variable_space_id	stage	log_xi	temperature	pressure	pH	start_date	complete_date
1	1	eq3	<null>	300.0	500.0	5.4541	2026-01-09 10:59:02.54	2026-01-09 10:59:02.55
2	1	eq6	-0.3000002024454176	300.0	500.0	5.3627	2026-01-09 10:59:02.55	2026-01-09 10:59:02.80

Example (With Paths):

```
SELECT
   id,
   variable_space_id,
   stage,
   log_xi,
   temperature,
   pressure,
   "pH",
   start_date,
   complete_date
 FROM equilibrium_space
 WHERE variable_space_id = 1
 ORDER BY stage, log_xi;
```

id	variable_space_id	stage	log_xi	temperature	pressure	pH	start_date	complete_date
1	1	eq3	<null>	300.0	500.0	5.4541	2026-01-09 11:02:59.49	2026-01-09 11:02:59.50
2	1	eq6	-inf	300.0	500.0	5.4536	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
3	1	eq6	-8.0	300.0	500.0	5.454	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
4	1	eq6	-7.0	300.0	500.0	5.4574	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
5	1	eq6	-6.443549140627881	300.0	500.0	5.4706	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
6	1	eq6	-6.0	300.0	500.0	5.5368	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
7	1	eq6	-5.0	300.0	500.0	6.0213	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
8	1	eq6	-4.734955647654595	300.0	500.0	6.1948	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
9	1	eq6	-4.0	300.0	500.0	6.5286	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
10	1	eq6	-3.8915466083537007	300.0	500.0	6.6473	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
11	1	eq6	-3.22983825514633	300.0	500.0	6.3563	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
12	1	eq6	-3.0	300.0	500.0	6.3854	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
13	1	eq6	-2.0	300.0	500.0	6.2304	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
14	1	eq6	-2.0	300.0	500.0	6.2304	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
15	1	eq6	-1.8955811793524053	300.0	500.0	5.6519	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
16	1	eq6	-1.4778975981455702	300.0	500.0	5.7453	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
17	1	eq6	-1.4727156101843601	300.0	500.0	5.6938	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
18	1	eq6	-1.3782070009986855	300.0	500.0	5.3294	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
19	1	eq6	-1.0	300.0	500.0	5.3303	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80
20	1	eq6	-0.3000002024454176	300.0	500.0	5.3627	2026-01-09 11:02:59.50	2026-01-09 11:02:59.80

equilibrium_reactants

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The equilibrium reactant identifier	
	✓		equilibrium_space_id	INTEGER	The id of the equilibrium space point	
			name	VARCHAR	The name of the reactant	
			affinity	DOUBLE	The affinity of the reactant	
			relative_rate	DOUBLE	The molar rate of reactant titration relative to the total molar quantity of all reactants	mol/mol
			log_moles_reacted	DOUBLE	The moles of reactant that have been reacted up to this ξ step	$\log mol$
			log_moles_remaining	DOUBLE	The moles of reactant that remain to be reacted	$\log mol$
			log_mass_reacted	DOUBLE	The mass of the reactant that have been reacted up to this ξ step	$\log kg$
			log_mass_remaining	DOUBLE	The mass of the reactant that remain to be reacted	$\log kg$

- INDEX equilibrium_reactants_equilibrium_space_id_name_idx on (equilibrium_space_id, name)
- INDEX equilibrium_reactants_affinity_idx on (affinity)
- INDEX equilibrium_reactants_log_moles_reacted_idx on (log_moles_reacted)
- INDEX equilibrium_reactants_log_moles_remaining_idx on (log_moles_remaining)
- INDEX equilibrium_reactants_log_mass_reacted_idx on (log_mass_reacted)
- INDEX equilibrium_reactants_log_mass_remaining_idx on (log_mass_remaining)

If an equilibrium_space entry represents a stage in which mass is transfered, (i.e. reacted or precipitated) and the system has N reactants, then the point will be associated with N rows in the equilibrium_reactants table, one for each reactant.

> **Note**: The exception to this rule is for fixed_gas_reactants, which the eleanor.kernel.eq36 kernel treats differently. Fixed gas reactants are enforced by requiring equilibrium with a fictitious solid phase in equilibrium_pure_solids, e.g. CO2(g) as a fixed gas reactant will produce a pure solid of "fix_fCO2(g)", and do not appear in the equilibrium_reactants table.

The equilibrium_reactants table is most useful for two purposes:

1. **Quality assurance.** At the end of the equilibration, all the log_moles_remaining ought to be -inf (no reactant). If that is not the case, then the reaction did not progress far enough and kernel settings likely need to be tweaked.
2. **Path Tracking.** If titration path tracking is enabled, the log_moles_reacted should monotonically increase while log_moles_remaining monotonically decreases along the path (ordered by log_xi from the equilibrium_space table).

Example:

```
SELECT
   id,
   equilibrium_space_id AS eid,
   name,
   affinity,
   relative_rate AS rate,
   log_moles_reacted AS moles_reacted,
   log_moles_remaining,
   log_mass_reacted,
   log_mass_remaining
 FROM equilibrium_reactants
 WHERE equilibrium_space_id = 2
 ORDER BY name;
```

```
+----+-----+------------------+-----------+------+----------------------+---------------------+----------------------+--------------------+
| id | eid | name             | affinity  | rate | moles_reacted        | log_moles_remaining | log_mass_reacted     | log_mass_remaining |
|----+-----+------------------+-----------+------+----------------------+---------------------+----------------------+--------------------|
| 5  | 2   | CaCl2            | 9999999.0 | 0.0  | -2.0                 | -inf                | 0.045283851395135605 | -inf               |
| 3  | 2   | CalciumHydroxide | 9999999.0 | 0.0  | -2.0                 | -inf                | -0.13021109763031236 | -inf               |
| 1  | 2   | hematite         | 1.43      | 1.0  | -0.29999760285774607 | -inf                | 1.903285375549671    | -inf               |
| 6  | 2   | N2(g)            | 9999999.0 | 1.0  | -0.29999760285774607 | -inf                | 1.1473671077937864   | -inf               |
| 2  | 2   | olivine-ss       | 1.7744    | 1.0  | -0.29999760285774607 | -inf                | 1.9517453891382763   | -inf               |
| 4  | 2   | Si               | 9999999.0 | 0.0  | -2.0                 | -inf                | -0.5515101084889142  | -inf               |
+----+-----+------------------+-----------+------+----------------------+---------------------+----------------------+--------------------+
```

equilibrium_elements

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The equilibrium element identifier	
	✓		equilibrium_space_id	INTEGER	The id of the equilibrium space point	
			name	VARCHAR	The name of the element (atomic symbol)	
			log_molality	DOUBLE	The molality of the element in solution	$\log molal$
			mass_fraction	DOUBLE	The fraction of solution mass accounted for by the element	unitless

- INDEX equilibrium_elements_equilibrium_space_id_name_idx on (equilibrium_space_id, name)
- INDEX equilibrium_elements_log_molality_idx on (log_molality)

Every equilibrium_space entry will be associated with some number of rows in the equilibrium_elements table, one for each element in the fluid. These rows record the name and log_molality of the element in the fluid and the mass_fraction of the fluid that element represents.

If no precipitates form, then these rows should agree with the entries in the `elements` table. However, that is rarely the case, so expect them to differ.

Example:

```
SELECT *
 FROM equilibrium_elements
 WHERE equilibrium_space_id = 2
 ORDER BY name;

+----+----------------------+------+---------------------+-----------------------+
| id | equilibrium_space_id | name | log_molality        | mass_fraction         |
|----+----------------------+------+---------------------+-----------------------|
| 16 | 2                    | C    | -5.5935330597128265 | 3.01624e-08           |
| 14 | 2                    | Ca   | -2.1571530866467845 | 0.000274913           |
| 12 | 2                    | Cl   | -1.6971649242940714 | 0.000701302           |
| 15 | 2                    | Fe   | -6.133895871377041  | 4.04134e-08           |
| 13 | 2                    | H    | 2.0457140589408676  | 0.110299              |
| 17 | 2                    | Mg   | -4.097879571162284  | 1.9108999999999998e-06 |
| 19 | 2                    | N    | 0.002835353178722796 | 0.0138865             |
| 18 | 2                    | Na   | -8.998192757497696  | 2.2738399999999998e-11 |
| 11 | 2                    | O    | 1.7443829627474854  | 0.8747969999999999    |
| 20 | 2                    | Si   | -2.838188943951825  | 4.01526e-05           |
+----+----------------------+------+---------------------+-----------------------+
```

equilibrium_aqueous_species

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The equilibrium aqueous species identifier	
	✓		equilibrium_space_id	INTEGER	The id of the equilibrium space point	
			name	VARCHAR	The name of the aqueous species, e.g. “H2O2”	
			log_molality	DOUBLE	The logarithm of the molality of that species in solution	log *molal*
			log_activity	DOUBLE	The logarithm of the activity of that species in solution	unitless
			log_gamma	DOUBLE	The logarithm of the activity coefficient of that species in solution	unitless

- INDEX equilibrium_aqueous_species_equilibrium_space_id_name_idx on (equilibrium_space_id, name)
- INDEX equilibrium_aqueous_species_log_molality_idx on (log_molality)
- INDEX equilibrium_aqueous_species_log_activity_idx on (log_activity)

Every equilibrium_space entry will be associated with some number of rows in the equilibrium_aqueous_species table, one for each species in the fluid. These rows record the name, log_molality, log_activity and log_gamma (log-activity coefficient) of the species in accordance with the activity model used by the kernel.

Example:

```
SELECT *
 FROM equilibrium_aqueous_species
 WHERE equilibrium_space_id = 2
 ORDER BY log_molality DESC, name
 LIMIT 10;

+----+----------------------+-------+--------------+--------------+-----------+
| id | equilibrium_space_id | name  | log_molality | log_activity | log_gamma |
|----+----------------------+-------+--------------+--------------+-----------|
| 65 | 2                    | N2    | -0.3092      | -0.3092      | 0.0       |
| 66 | 2                    | NH3   | -1.7195      | -1.7195      | 0.0       |
| 67 | 2                    | Cl-   | -1.7863      | -1.9078      | -0.1215   |
| 68 | 2                    | NH4+  | -2.2149      | -2.3465      | -0.1316   |
| 69 | 2                    | CaCl+ | -2.4406      | -2.5666      | -0.126    |
| 70 | 2                    | Ca+2  | -2.4947      | -2.9493      | -0.4545   |
| 71 | 2                    | SiO2  | -2.8385      | -2.8385      | 0.0       |
| 72 | 2                    | H2    | -3.1232      | -3.1179      | 0.0052    |
| 73 | 2                    | CaOH+ | -4.0071      | -4.1331      | -0.126    |
| 74 | 2                    | Mg+2  | -4.3002      | -4.7213      | -0.4211   |
+----+----------------------+-------+--------------+--------------+-----------+
```

equilibrium_gases

PK	FK	N	Column Name	Type	Description	Unit
✓			`id`	`INTEGER`	The equilibrium gas identifier	
	✓		`equilibrium_space_id`	`INTEGER`	The id of the equilibrium space point	
			`name`	`VARCHAR`	The name of the gas, e.g. “H2(g)”	
			`log_fugacity`	`DOUBLE`	The logarithm of the fugacity of the gas at this ξ step	$\log bar$

- INDEX `equilibrium_gases_equilibrium_space_id_name_idx` on (`equilibrium_space_id`, `name`)
- INDEX `equilibrium_gases_log_fugacity_idx` on (`log_fugacity`)

Every `equilibrium_space` entry will be associated with some number of rows in the `equilibrium_gases` table, one for each gas phase. These rows record the `name` and `log_fugacity`.

Example:

```
SELECT *
 FROM equilibrium_gases
 WHERE equilibrium_space_id = 2
 ORDER BY log_fugacity DESC, name;

+----+----------------------+--------+--------------+
| id | equilibrium_space_id | name   | log_fugacity |
|----+----------------------+--------+--------------|
| 1  | 2                    | N2(g)  | 2.41361      |
| 2  | 2                    | H2O(g) | 1.85299      |
| 3  | 2                    | H2(g)  | -0.58253     |
| 4  | 2                    | NH3(g) | -0.83984     |
| 5  | 2                    | CO2(g) | -3.5         |
| 6  | 2                    | CH4(g) | -4.06719     |
| 7  | 2                    | CO(g)  | -7.60332     |
| 8  | 2                    | NO(g)  | -23.50603    |
| 9  | 2                    | N2O(g) | -26.10458    |
| 10 | 2                    | O2(g)  | -34.26937    |
+----+----------------------+--------+--------------+
```

equilibrium_pure_solids

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The equilibrium pure solid id	
	✓		equilibrium_space_id	INTEGER	The id of the equilibrium space point	
			name	VARCHAR	The name of the pure solid, e.g. “fayalite”	
			log_qk	DOUBLE	The saturation index of the pure solid phase	unitless
			affinity	DOUBLE	the affinity of the pure solid	$kcal$
			log_moles	DOUBLE	The logarithm of the net moles of the solid that have precipitated up to this ξ step	$\log mol$
			log_mass	DOUBLE	The logarithm of the net mass of the solid that has precipitated up to this ξ step	$\log g$
			log_volume	DOUBLE	The logarithm of the volume of the solid that has precipitated	$\log cm^3$

- INDEX equilibrium_pure_solids_equilibrium_space_id_name_idx on (equilibrium_space_id, name)
- PARTIAL INDEX equilibrium_pure_solids_log_moles_present_idx on (equilibrium_space_id) WHERE log_moles > '-Infinity'::double precision
- INDEX equilibrium_pure_solids_affinity_idx on (affinity)
- INDEX equilibrium_pure_solids_log_qk_idx on (log_qk)

Every equilibrium_space entry will be associated with some number of rows in the equilibrium_pure_solids table, one for each pure solid (not a solid solution). These rows record the name, saturation index (log_qk), affinity, log_moles, log_mass and log_volume of the solid phase.

Notes:

1. The solids that appear in this table include both solids that form as well as those that *could* have formed based on the composition of the system. For example, the solids associated with the `"eq3"` stage will all be hypothetical as the kernel does not partition mass in that stage. As such, the value in the `log_moles`, `log_mass` and `log_volume` columns will be `-inf`. Similarly, those columns will be `-inf` for `"eq6"`-stage equilibrium space points if the phase does not actually precipitate.
2. In many cases, volume data is lacking. Even if a solid precipitates, it will *likely* have a $V = 0$ (`log_volume = -inf`).

Example:

```
SELECT *
 FROM equilibrium_pure_solids
 WHERE equilibrium_space_id = 2
 ORDER BY log_mass DESC, log_qk DESC, name
 LIMIT 10;
```

```
+-----+----------------------+------------+----------+----------+-----------+--------------------+------------+
| id  | equilibrium_space_id | name       | log_qk   | affinity | log_moles | log_mass           | log_volume |
|-----+----------------------+------------+----------+----------+-----------+--------------------+------------|
| 56  | 2                    | magnetite  | -0.0     | -0.0     | -0.2891   | 2.0755104645244136 | -inf       |
| 62  | 2                    | antigorite | 1.67537  | 4.39389  | -inf      | -inf               | -inf       |
| 110 | 2                    | tremolite  | -0.0     | -0.0     | -inf      | -inf               | -inf       |
| 109 | 2                    | talc       | -0.12292 | -0.32237 | -inf      | -inf               | -inf       |
| 68  | 2                    | chrysotile | -0.30052 | -0.78816 | -inf      | -inf               | -inf       |
| 74  | 2                    | diopside   | -0.42799 | -1.12247 | -inf      | -inf               | -inf       |
| 57  | 2                    | hematite   | -0.54526 | -1.43002 | -inf      | -inf               | -inf       |
| 78  | 2                    | enstatite  | -0.739   | -1.93813 | -inf      | -inf               | -inf       |
| 105 | 2                    | quartz     | -0.8933  | -2.34282 | -inf      | -inf               | -inf       |
| 88  | 2                    | goethite   | -0.90359 | -2.36978 | -inf      | -inf               | -inf       |
+-----+----------------------+------------+----------+----------+-----------+--------------------+------------+
```

equilibrium_solid_solutions

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The equilibrium solid solution identifier	
	✓		equilibrium_space_id	INTEGER	The id of the equilibrium space point	
			name	VARCHAR	The name of the solid solution, e.g. “olivine”	
			log_qk	DOUBLE	The saturation index of the solid solution	unitless
			affinity	DOUBLE	The affinity of the solid solution	$kcal$
			log_moles	DOUBLE	The logarithm of the net moles of the solid solution that have precipitated up to this ξ step	$\log mol$
			log_mass	DOUBLE	The logarithm of the net mass of the solid solution that has precipitated up to this ξ step	$\log g$

PK	FK	N	Column Name	Type	Description	Unit
			log_volume	DOUBLE	The logarithm of the volume of the solid solution that has precipitated	$\log cm^3$

- INDEX equilibrium_solid_solutions_equilibrium_space_id_name_idx on (equilibrium_space_id, name)
- PARTIAL INDEX equilibrium_solid_solutions_log_moles_present_idx on (equilibrium_space_id) WHERE log_moles > '-Infinity'::double precision
- INDEX equilibrium_solid_solutions_affinity_idx on (affinity)
- INDEX equilibrium_solid_solutions_log_qk_idx on (log_qk)

Every equilibrium_space entry will be associated with some number of rows in the equilibrium_solid_solutions table, one for each solid solution. These rows record the name, saturation index (log_qk), affinity, log_moles, log_mass and log_volume of the solid solution. Additionally, the end member composition of the solid solutions are stored in the equilibrium_end_members auxiliary table.

Notes:

1. The solid solutions that appear in this table include both solid solutions that form as well as those that *could* have formed based on the composition of the system. For example, the solid solutions associated with the "eq3" stage will all be hypothetical as the kernel does not partition mass in that stage. As such, the value in the log_moles, log_mass and log_volume columns will be -inf. Similarly, those columns will be -inf for "eq6"-stage equilibrium space points if the phase does not actually precipitate.
2. In many cases, volume data is lacking. Even if a solid precipitates, it will *likely* have a $V = 0$ (log_volume = -inf).

Example:

```
SELECT *
 FROM equilibrium_solid_solutions
 WHERE equilibrium_space_id = 2
 ORDER BY log_mass DESC, log_qk DESC, name
 LIMIT 10;
```

id	equilibrium_space_id	name	log_qk	affinity	log_moles	log_mass	log_volume
13	2	talc-ss	-0.0	-0.0	-1.0435	1.5450472044011843	-inf
12	2	serpentine	-0.0	-0.0	-1.2932	1.1838390370564211	-inf
11	2	amphib_ternary-ss	-0.0	-0.0	-2.1849	0.7248327204665614	-inf
19	2	ca-amphib_binary-ss	-0.0	-0.0	-inf	-inf	-inf
18	2	clinopyroxene-ss	-0.41699	-1.09361	-inf	-inf	-inf
17	2	orthopyroxene-ss	-0.59034	-1.54825	-inf	-inf	-inf
16	2	ideal_olivine-ss	-0.64149	-1.6824	-inf	-inf	-inf
14	2	olivine-ss	-0.64149	-1.6824	-99999.0	-inf	-inf
20	2	brucite-ss	-1.2999	-3.40916	-inf	-inf	-inf
15	2	carbonate-ss	-4.07608	-10.69009	-inf	-inf	-inf

equilibrium_end_members

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The equilibrium solid solution end member identifier	
	✓		equilibrium_solid_solution_id	INTEGER	The id of the equilibrium solid solution	
			name	VARCHAR	The name of the end member, e.g. “fayalite”	
			log_qk	DOUBLE	The saturation index of the end member	unitless
			affinity	DOUBLE	The affinity of the end member	$kcal$
			log_moles	DOUBLE	The logarithm of the net moles of the end member that have precipitated up to this ξ step	$\log mol$
			log_mass	DOUBLE	The logarithm of the net mass of the end member that has precipitated up to this ξ step	$\log g$
			log_volume	DOUBLE	The logarithm of the volume of the end member that has precipitated	$\log cm^3$

- INDEX `equilibrium_end_members_equilibrium_solid_solution_id_name_idx` on (`equilibrium_solid_solution_id`, `name`)
- PARTIAL INDEX `equilibrium_end_members_log_moles_present_idx` on (`equilibrium_solid_solution_id`) WHERE `log_moles > '-Infinity'::double precision`
- INDEX `equilibrium_end_members_affinity_idx` on (`affinity`)
- INDEX `equilibrium_end_members_log_qk_idx` on (`log_qk`)

Every `equilibrium_solid_solutions` entry will be associated with some number of rows in the `equilibrium_end_members` table, one for each end member. These rows record the `name`, saturation index (`log_qk`), `affinity`, `log_moles`, `log_mass` and `log_volume` of the solid phase.

Notes:

1. The end members that appear in this table include both end members that form as well as those that *could* have formed based on the composition of the system. For example, end members of solid solutions associated with the `"eq3"` stage will all be hypothetical as the kernel does not partition mass in that stage. As such, the value in the `log_moles`, `log_mass` and `log_volume` columns will be `-inf`. Similarly, those columns will be `-inf` for `"eq6"`-stage equilibrium space points if the phase does not actually precipitate.
2. In many cases, volume data is lacking. Even if a solid precipitates, it will *likely* have a $V = 0$ (`log_volume = -inf`).

Example:

```
SELECT *
 FROM equilibrium_end_members
 WHERE equilibrium_solid_solution_id = 13
 ORDER BY log_mass DESC, log_qk DESC, name;

+----+-------------------------------+--------------+--------+----------+-----------+--------------------+------------+
| id | equilibrium_solid_solution_id | name         | log_qk | affinity | log_moles | log_mass           | log_volume |
|----+-------------------------------+--------------+--------+----------+-----------+--------------------+------------|
| 28 | 13                            | talc         | -0.0   | -0.0     | -1.0845   | 1.4944328987263986 | -inf       |
```

```
| 29 | 13                           | minnesotaite | -0.0   | -0.0     | -2.0892   | 0.5865197925102743 | -inf       |
+----+------------------------------+--------------+--------+----------+-----------+--------------------+------------+
```

equilibrium_redox_reactions

PK	FK	N	Column Name	Type	Description	Unit
✓			id	INTEGER	The equilibrium redox reaction identifier	
	✓		equilibrium_space_id	INTEGER	The id of the equilibrium space point	
			couple	VARCHAR	The name of the redox couple	
			"Eh"	DOUBLE	The redox potential	V
			pe	DOUBLE	The negative logarithm of the hypothetical electron activity	unitless
			"log_fO2"	DOUBLE	The logarithm of the oxygen (O_2) fugacity	$\log bar$
			"Ah"	DOUBLE	The redox affinity	$kcal$

- INDEX equilibrium_redox_reactions_equilibrium_space_id_couple_idx on (equilibrium_space_id, couple)
- INDEX equilibrium_redox_reactions_eh_idx on (Eh)
- INDEX equilibrium_redox_reactions_log_fo2_idx on (log_fO2)

Each equilibrium_space entry can be associated with one or more rows in the equilibrium_redox_reactions table, one for each redox constraint on the system.

> **Note**: In the present version of **Eleanor**, there will typically a single equilibrium_redox_reactions entry for each equilibrium_space entry, and the values in this table will be wholly redundant with equilibrium_space.

Example:

```
SELECT redox.*
 FROM equilibrium_redox_reactions AS redox
 JOIN equilibrium_space AS es ON es.id = redox.equilibrium_space_id AND es.variable_space_id = 1;

+----+----------------------+---------+--------+---------+---------+---------+
| id | equilibrium_space_id | couple  | Eh     | pe      | log_fO2 | Ah      |
|----+----------------------+---------+--------+---------+---------+---------|
| 1  | 1                    | DEFAULT | 0.373  | 3.2836  | -1.0    | 8.612   |
| 2  | 2                    | DEFAULT | -0.562 | -4.9403 | -34.269 | -12.957 |
+----+----------------------+---------+--------+---------+---------+---------+
```

Appendix - Example Order

```yaml
name: A Wild Example Order
creator: 39 Alpha Research, Team One
date: 2026-01-09
notes: This is a nonsensical order intended to demonstrate a wide

kernel:
  kind: eq36
  model: b-dot
  charge_balance: H+
  track_paths: true
  eq6_config:
    xi_max: 1
    print_step_interval: 1

elements:
  C: -5
  Mg: -9
  Fe: -9
  Na: [-4, -5]
  Si:
    min: -9
    max: -6
  Ca:
    mean: -4
    stddev: 0.01
    min: -5
    max: -3

species:
  H+: -7
  O2(g): -1
reactants:
  CaCl2:
    type: aqueous
    amount: -3.0
    titration_rate: 1.0
  Na:
    type: element
    amount: -4.0
    titration_rate: 1.0
  CO2(g):
    type: fixed gas
    amount: -0.3
    log_fugacity: -3.5
  N2(g):
    type: gas
    amount: -0.3
    titration_rate: 1.0
  hematite:
    type: mineral
    amount: -0.3
    titration_rate: 1.0
  olivine-ss:
    type: solid solution
    amount: -0.3
    titration_rate: 1.0
    end_members:
      fayalite: 0.6
      forsterite: 0.4
  CalciumHydroxide:
    type: special
    amount: -0.3
    titration_rate: 1.0
    composition:
      Ca: 1
      O: 2
      H: 2

suppressions:
  - METHANE
  - antigorite
  - type: mineral
    except:
      - magnetite
```